database or from some remote service, such as an `ICurrencyProvider`.

This situation, in which you need a configured service to configure your `IOptions<T>`, is potentially hard to resolve. Remember that you declared your `IOptions<T>` configuration as part of your app's DI configuration. But if you need to resolve a service from DI to configure the `IOptions` object, you're stuck with a chicken-and-egg problem: how can you access a service from the DI container before you've finished configuring the DI container?

This circular problem has several potential solutions, but the easiest approach is to use an alternative API for configuring `IOptions` instances, using the `OptionsBuilder<T>` type. This type is effectively a wrapper around some of the core `IOptions` interfaces, but it often results in a terser and more convenient syntax to the approach you've seen so far.

TIP Another helpful feature of `OptionsBuilder<T>` is adding validation to your `IOptions` objects. This ensures that your configuration is loaded and bound correctly on app startup so that you don't have any typos in your configuration section names, for example. You can read more about adding validation to your `IOptions` objects on my blog at http://mng.bz/qrjJ.

The following listing shows the equivalent of listing 31.12 but using `OptionsBuilder<T>` instead. You create an `OptionsBuilder<T>` instance by calling `AddOptions<T>()`, and then chain additional methods such as `BindConfiguration()` and `Configure()` to configure your final `IOptions<T>` object,